数据结构基础

期末复习讲义

知识点系统提炼（中文） + 历年期末真题分章巩固（英文）

涵盖第 2–9 章 · 真题含 2015–2025 学年期末（模拟）卷

本讲义按“**核心知识点** → **真题巩固**”组织。知识点用中文讲解，**做题需用到的术语保留英文**；每章末附对应章节历年真题（判断 / 选择 / 程序填空 / 编程），**英文原题**、**不含答案**，供自测使用。

2026 年 6 月 21 日

目录

使用说明 **2**

1 专题一 算法分析（Algorithm Analysis） **3**
1.1 核心知识点 3
1.2 真题巩固 4

2 专题二 线性表、栈与队列（Lists, Stacks & Queues） **7**
2.1 核心知识点 7
2.2 真题巩固 8

3 专题三 二叉树与搜索树（Binary Trees & Search Trees） **11**
3.1 核心知识点 11
3.2 真题巩固 12

4 专题四 优先队列与堆（Priority Queues / Heaps） **16**
4.1 核心知识点 16
4.2 真题巩固 16

5 专题五 排序（Sorting） **19**
5.1 核心知识点 19
5.2 真题巩固 20

6 专题六 散列表（Hashing） **23**
6.1 核心知识点 23
6.2 真题巩固 24

7 专题七 并查集（Disjoint Set / Union-Find） **27**
7.1 核心知识点 27
7.2 真题巩固 27

8 专题八 图（Graph Algorithms） **29**
8.1 核心知识点 29
8.2 真题巩固 30

使用说明

- **结构**：全书分 8 个专题，对应教材第 2–9 章。每个专题先给出**核心知识点**（定义、性质、复杂度、算法要点、易错点），再附**真题巩固**。
- **知识点语言**：用中文讲解，但**考试 / 做题会用到的专业术语保留英文**（如 *Big-O*, *percolate up/down*, *quadratic probing*, *topological sort* 等），方便对接英文试卷。
- **真题**：选自 2015–2025 历年期末（模拟）卷与 2023–2025 正式期末卷，按章节归类，**保留英文原题、均不附答案**，请独立作答后再对照教材自查。
- **颜色约定**：蓝框 = 核心要点，红框 = 易错点，绿框 = 解题技巧。

“考点高度稳定——抓住每种结构的性质、操作复杂度与典型陷阱，英文真题自然会做。”

1 专题一　算法分析（Algorithm Analysis）

1.1 核心知识点

1. 算法的五个特性

算法 (algorithm) 是一组有限指令，必须满足：**输入 (input)**（0 个或多个）、**输出 (output)**（至少 1 个）、**确定性 (definiteness)**、**有穷性 (finiteness)**、**有效性 (effectiveness)**。程序 (program) 不要求有穷性（如操作系统长期运行）。

2. 分析什么

关注**时间复杂度 (time complexity)** 与**空间复杂度 (space complexity)**：输入规模 N 增大时的增长趋势，而非绝对耗时。常分析**最坏情况 (worst case)** $T_{\text{worst}}(N)$ 与**平均情况 (average case)** $T_{\text{avg}}(N)$。

3. 渐进记号 (Asymptotic Notation)

四种渐进记号

- $T(N) = O(f(N))$：存在 $c, n_0 > 0$ 使 $N \geq n_0$ 时 $T(N) \leq c\, f(N)$——**上界 (upper bound)**，取最小有意义上界。
- $T(N) = \Omega(g(N))$：$N \geq n_0$ 时 $T(N) \geq c\, g(N)$——**下界 (lower bound)**，取最大有意义下界。
- $T(N) = \Theta(h(N))$：同时 $O(h(N))$ 与 $\Omega(h(N))$——**同阶 (tight bound)**。
- $T(N) = o(p(N))$：是 $O(p(N))$ 但非 $\Theta(p(N))$——**严格低阶**。

运算规则：$T_1 + T_2 = \max(O(f), O(g))$；$T_1 \cdot T_2 = O(f \cdot g)$；$k$ 次多项式为 $\Theta(N^k)$；对任意常数 k 有 $\log^k N = O(N)$。
常见增长率（由慢到快）：$1 < \log N < N < N \log N < N^2 < N^3 < 2^N < N!$

4. 一般分析规则

循环 (loop)：体内时间 × 迭代次数；嵌套循环 (nested loop)：相乘；连续语句：相加取最大项；`if/else`：测试时间 + 较大分支；递归 (recursion)：列**递推式 (recurrence)** 再求解。朴素递归 Fibonacci 因重复计算呈**指数级**。

5. 最大子序列和 (Maximum Subsequence Sum)——同一问题四种算法

算法	思想	复杂度
1	枚举起点、终点并重新求和	$O(N^3)$
2	枚举起点、终点并递增维护和	$O(N^2)$
3	分治 (divide and conquer)：左 / 右 / 跨中点	$O(N\log N)$
4	在线 (on-line) 扫描，负前缀和清零	$O(N)$

分治递推 $T(N)=2T(N/2)+cN\Rightarrow O(N\log N)$。算法 4 是 **on-line algorithm**（任一时刻对已读入数据都能给出当前答案）。

6. 运行时间中的对数

- **二分查找 (Binary Search)**：有序静态表，每次折半，$O(\log N)$。
- **欧几里得算法 (Euclid's Algorithm)**：$\gcd(m,n)=\gcd(n,m \bmod n)$，$O(\log\min(m,n))$。
- **快速幂 (Exponentiation by Squaring)**：$X^N=(X^{N/2})^2[\cdot X]$，$O(\log N)$。

复杂度判定技巧

倍增法：若 $T(2N)/T(N)\approx 2,4,8$，则分别约为 $O(N),O(N^2),O(N^3)$。比较两算法须保证 N 足够大（如 10^6N 与 N^2，当 $N<10^6$ 时线性反而更慢）。

1.2　真题巩固

真题巩固 Exercises（英文原题，无答案）

[True or False]

1. 〔24-25〕 $114514^N=O(N!)$. (T　F)

2. 〔23-24〕 $\log(N!)=\Omega(N\cdot\log N)$. (T　F)

3. 〔18-19〕 $\log N^{20}$ is $O(N)$. (T　F)

4. 〔17-18〕 $O(N^2)$ is the same as $O(1+2+3+\cdots+N)$. (T　F)

5. 〔20-21〕 Consider two programs with time complexities $T_1=O(2^n)$ and $T_2=O(n)$. Then program 2 must run faster than program 1. (T　F)

6. 〔15-16〕 The Fibonacci sequence is $F_0=0,F_1=1,F_N=F_{N-1}+F_{N-2}$. The time complexity of the function that calculates F_N **iteratively** is $\Theta(F_N)$. (T　F)

[Single Choice]

1. 〔16-17〕 For the following piece of code, the lowest upper bound of the time complexity is:

```
if ( A > B ) {
  for ( i=0; i<N*2; i++ )
    for ( j=N*N; j>i; j-- )  C += A;
} else {
  for ( i=0; i<N*N/2; i++ )
    for ( j=N; j>i; j-- )
      for ( k=0; k<N*100; k++ )  C += B;
}
```

A. $O(N^3)$
B. $O(N^4)$
C. $O(N^5)$
D. $O(N^6)$

2. 〔16-17〕 Given four algorithms with their runtimes for problem size 100 and their time complexities. Which algorithm is the fastest for problem size 200?

Algorithm	Runtime (size 100)	Time Complexity
A	100	$O(N)$
B	50	$O(N^2)$
C	25	$O(N^3)$
D	10	$O(N^4)$

A. A
B. B
C. C
D. D

3. 〔20-21〕 Suppose A is an array of length N with random numbers. What is the worst-case time complexity of the following program?

```
void function( int A[], int N ) {
    int i, j = 0, cnt = 0;
    for (i = 0; i < N; ++i) {
        for (; j < N && A[j] <= A[i]; ++j);
        cnt += j - i;
    }
}
```

A. $O(N^2)$
B. $O(N \log N)$

C. $O(N)$

D. $O(N^{1.5})$

4. 〔15-16〕 To judge whether an integer $N\,(>10)$ is prime, we check if it is divisible by any odd number from 3 to $\sqrt{N}$. The time complexity of this algorithm is:

A. $O(N/2)$

B. $O(\sqrt{N})$

C. $O(\sqrt{N}\log N)$

D. $O(0.5\log N)$

2 专题二　线性表、栈与队列（Lists, Stacks & Queues）

2.1　核心知识点

1. 抽象数据类型 (Abstract Data Type, ADT)

ADT 将对象与操作的**规格说明 (specification)** 与其**表示 / 实现 (implementation)** 分离——只关心“能做什么”，不关心“怎么做”。

2. 线性表 (List) 的两种实现

操作	顺序存储 (array)	链式存储 (linked list)	说明
按位查找 `Find_Kth`	$O(1)$	$O(N)$	数组可随机访问
插入 / 删除（已知位置）	$O(N)$	$O(1)$	数组需搬移元素
空间	预分配 MaxSize	随用随分配	链表多耗指针

双向循环链表 (doubly linked circular list)：满足 `p==p->llink->rlink==p->rlink->llink`，便于找前驱、删除当前结点，常加**头结点 (dummy head)** 统一边界。**游标实现 (cursor implementation)**：无指针语言中用结构数组 + 空闲链表模拟链表。

3. 栈 (Stack，LIFO)

仅在栈顶 (top) 插入 (`Push`) / 删除 (`Pop`)。典型应用：

- **平衡符号 / 括号匹配 (balancing symbols)**：左符入栈、右符弹出匹配，$O(N)$ 在线算法。
- **后缀表达式 (postfix / Reverse Polish) 求值**：操作数入栈，遇运算符弹出计算，$O(N)$，无需优先级。
- **中缀转后缀 (infix to postfix)**：操作数直接输出；运算符入栈，遇更低优先级弹栈；右括号弹至左括号。注意 in-stack 与 incoming 优先级、减法左结合、乘方右结合。
- **函数调用**：系统栈保存栈帧 (stack frame)；尾递归可优化为循环。

4. 队列 (Queue，FIFO)

一端入队 (`Enqueue`)、另一端出队 (`Dequeue`)。数组实现用**循环队列 (circular queue)**：`Rear=(Rear+1)%Capa`

循环队列满 / 空判定

仅用 `front` 与 `rear` 时需**空出一格**区分满空（容量 $m-1$）；若额外维护 `size` 字段或标志位，则可用满全部 m 格。

易错点

- 合法出栈序列：入栈序列固定时并非任意排列都能得到（如 `abcde` 不能得到 `cedab`）。
- “先到先服务 (FCFS)”用**队列**实现，不是栈。

- 顺序表删除**首**元素 $O(N)$（要搬移），插入 / 删除**尾**元素才是 $O(1)$。

2.2　真题巩固

真题巩固 Exercises（英文原题，无答案）

[True or False]

1. 〔18-19〕 If keys are pushed onto a stack in the order `abcde`, then it is impossible to obtain the output sequence `cedab`. (T　F)

2. 〔20-21〕 In most restaurants we follow the principle "First come, first served". This principle can be implemented by a queue. (T　F)

3. 〔16-17〕 An algorithm to check for balancing symbols in an expression uses a queue to store the partial expression. (T　F)

4. 〔17-18〕 For a sequentially stored linear list of length N, the time complexities for deleting the first element and inserting the last element are $O(1)$ and $O(N)$, respectively. (T　F)

5. 〔17-18〕 If the most commonly used operations are to visit a random position and to insert / delete the last element, then sequential storage works the fastest. (T　F)

6. 〔23-24〕 Suppose we have a doubly linked circular list with N regular nodes and a dummy head. Given `p` pointing to an arbitrary node, we can get back to the original position by executing "`p = p->prev;`" $N+1$ times. (T　F)

[Single Choice]

1. 〔24-25〕 In a doubly linked circular list, how many pointers must be updated to insert q after p?

A. 1
B. 2
C. 3
D. 4

2. 〔24-25〕 Design an algorithm to determine whether the parentheses `()` in an expression are properly matched. Which data structure is the most suitable?

A. Sequentially stored linear list
B. Queue
C. Stack
D. Heap

3. 〔18-19〕 The following C-like function takes a Queue as argument. What does it do?

```
void foo(Queue Q) {
    Stack S = CreateStack();
    while (!IsEmpty(Q)) Push(S, Dequeue(Q));
    while (!IsEmpty(S)) Enqueue(Q, Pop(S));
    DisposeStack(S);
}
```

A. Removes the last item from Q
B. Keeps Q unchanged
C. Makes Q empty
D. Reverses Q

4. 〔23-24〕 Using a stack to convert the infix expression `((A+B)-C*(D/E))+F` to postfix, what is the maximum number of **operators** in the stack during conversion?

A. 2
B. 4
C. 3
D. 5

5. 〔23-24〕 The postfix expression of `a+(b-c)*d/e` is:

A. `abc-*de/+`
B. `abc-d*e/+`
C. `abc-de/*+`
D. `abc-de*/+`

6. 〔18-19〕 To convert the infix expression `4 * 3 + (6 * 3 - 12)` to postfix using a stack S, the minimum size of S must be:

A. 2
B. 3
C. 4
D. 5

7. 〔17-18〕 Given that the pushing sequence of a stack is $\{1, 2, \cdots, n\}$ and the popping sequence is $\{p_1, p_2, \cdots, p_n\}$. If $p_2 = n$, how many different possible popping sequences can we obtain?

A. 1
B. 2
C. $n - 1$
D. n

8. 〔17-18〕 An array of size m stores a circular queue. If the head pointer `front` and the current size variable `size` are used to represent the queue (instead of `front` and `rear`), then the maximum capacity can be:

A. $m-1$
B. m
C. $m+1$
D. cannot be determined

9. 〔15-16〕 If the pushing sequence of a stack is $\{1,2,3,\ldots,N\}$ and the first number popped out is i, then the j-th number popped must be:

A. $i-j-1$
B. $i-j$
C. $j-i-1$
D. cannot be determined

10. 〔15-16〕 A polynomial is represented by a linked list of its non-zero terms. Two polynomials have N_1 and N_2 non-zero terms, with highest exponents M_1 and M_2. The time complexity for multiplying them is:

A. $O(M_1 \times M_2)$
B. $O(N_1 \times N_2)$
C. $O(M_1 + M_2)$
D. $O(N_1 + N_2)$

3 专题三　二叉树与搜索树（Binary Trees & Search Trees）

3.1 核心知识点

1. 基本术语

结点的**度 (degree)** = 子树个数；**深度 (depth)**（根为 0 向下递增）；**高度 (height)**（叶为 0 向上递增）；含 N 结点的树有 $N-1$ 条边。

2. 二叉树性质

必记性质

- 第 i 层最多 2^{i-1} 个结点；高度（层数）为 k 的满二叉树最多 2^k-1 个结点。
- **叶子与度关系**：$n_0 = n_2 + 1$（n_0 叶子数，n_2 度为 2 的结点数）。
- 度题通用式：边数 $= \sum$(度 $\times$ 该度结点数) $= N-1$。

FirstChild-NextSibling：普通树转二叉树的标准表示（左指针指长子，右指针指右兄弟）。

3. 遍历 (Traversal) 与还原

- 先序 preorder（根左右）、中序 inorder（左根右）、后序 postorder（左右根）、层序 level-order（队列实现，类似 BFS）。
- **还原**：已知 **inorder + preorder** 或 **inorder + postorder** 可唯一确定二叉树；仅 preorder + postorder **不能**唯一确定。
- 表达式树 (expression tree)：inorder 得中缀、postorder 得后缀、preorder 得前缀。

4. 线索二叉树 (Threaded Binary Tree)

n 个结点有 $n+1$ 个空指针；中序线索化用空左指针存**中序前驱 (inorder predecessor)**、空右指针存**中序后继 (inorder successor)**。

5. 二叉搜索树 (Binary Search Tree, BST)

左子树键 $<$ 根 $<$ 右子树键，且递归成立。

- 查找 / 插入 / `FindMin`(一直向左) / `FindMax`(一直向右)：均 $O(h)$。
- **删除 (Delete)**：(1) 叶子直接删；(2) 单孩子用孩子顶替；(3) 双孩子用**右子树最小**（或左子树最大）顶替再删之。
- **中序遍历 BST 得递增序列**；高度依赖插入顺序，最坏退化为链 $O(N)$。

6. 线段树 (Segment Tree)

完全二叉树存于数组（结点 `node` 孩子为 `2*node`, `2*node+1`），存区间聚合值。建树 $O(N)$；**区间查询 (range query)** 与**单点更新 (point update)** 均 $O(\log N)$；区间修改用**懒标记 (lazy propagation)**。

易错点

- “BST 中两键同层 ⇒ 其中间值必是它们的父结点” **错**！同层只说明分处某公共祖先两侧。
- preorder 与 postorder **恰好相反** $\iff$ 每个结点至多一个孩子（退化为单链）。
- 普通树 T 转二叉树 BT：BT 的 preorder $=T$ 的 preorder；BT 的 inorder $=T$ 的 postorder。

3.2　真题巩固

真题巩固 Exercises（英文原题，无答案）

[True or False]

1. 〔24-25〕 If a complete binary tree has $n+1$ nodes in the last level and n nodes in the second last level, then the total number of nodes must be $3n$. (T　F)

2. 〔24-25〕 If the five integers $\{1,2,3,4,5\}$ are inserted into an initially empty BST in any order **starting with 3**, the resulting trees will all have the same height. (T　F)

3. 〔23-24〕 For a binary search tree (BST), its in-order traversal gives nodes in non-decreasing order. (T　F)

4. 〔23-24〕 Every node in the tree is the root of some subtree. (T　F)

5. 〔18-19〕 If the inorder and the postorder traversal sequences of a binary tree have exactly the same order, then none of the nodes in the tree has a right subtree. (T　F)

6. 〔17-18〕 If the preorder and the postorder traversal sequences of a binary tree have exactly the opposite orders, then none of the nodes in the tree has two subtrees. (T　F)

7. 〔17-18〕 Given a BST with 20 integer keys including 10, 11, 12, if 10 and 12 are on the same level, then 11 must be their common ancestor. (T　F)

8. 〔15-16〕 There exists a binary tree with 2016 nodes in total, and with 16 nodes having only one child. (T　F)

[Single Choice]

1. 〔24-25〕 Given a tree of degree 3, with 49 nodes of degree 2 and 25 nodes of degree 3, how many leaf nodes are there?

A. 101

B. 100

C. 150

D. 50

2. 〔24-25〕 Which of the following statements is correct regarding a binary search tree storing unique integers?

A. The integers 8 and 9 may be at the same level.

B. If 8 and 10 are at the same level, then 9 must be their parent node.

C. If 8 and 10 are at the same level, then 9 must not be their parent node.

D. None.

3. 〔24-25〕 For a binary tree, if its post-order sequence is $\{1, 3, 2, 6, 5, 7, 4\}$ and its in-order sequence is $\{1, 2, 3, 4, 5, 6, 7\}$, which statement is TRUE?

A. It is a complete binary tree

B. 6 is the parent of 5

C. It is a binary search tree

D. 3 is the parent of 2

4. 〔24-25〕 For an in-order threaded binary tree, if the post-order and in-order sequences are FDEBGCA and FDBEACG respectively, which pair of nodes' **left** links are both threads?

A. E and G

B. B and G

C. C and D

D. D and E

5. 〔24-25〕 Perform on an initially empty BST: Insert 50,30,70,20,40,60,80; Delete 50 (replaced with its in-order successor); Insert 55. Which node is the in-order predecessor of 60 in the final BST?

A. 30

B. 40

C. 55

D. The in-order predecessor of 60 does not exist

6. 〔23-24〕 If a binary search tree of N nodes is complete, which statement is FALSE?

A. The median node must be either the root or in the left subtree

B. The minimum key must be at a leaf node

C. The average search time for all nodes is $O(\log N)$

D. The maximum key must be at a leaf node

7. 〔23-24〕 Given a binary tree with in-order sequence $\{4, 1, 3, 8, 7, 12, 11, 9\}$ and post-order sequence $\{4, 1, 8, 12, 7, 9, 11, 3\}$. What is the index of 8 if we number the nodes in level-order

(root is 1)?

A. 5
B. 2
C. 3
D. 7

8. 〔18-19〕 When inserting a new key K into a BST with 512 nodes, the worst-case number of comparisons between K and the keys already in T is in the range of:

A. $[10, 511]$
B. $[9, 511]$
C. $[9, 512]$
D. $[10, 512]$

9. 〔17-18〕 Which one of the following is a possible postorder traversal sequence of a binary search tree?

A. 3 1 6 2 5 8 9 7 4
B. 3 1 2 6 5 8 9 7 4
C. 1 3 2 6 5 8 9 7 4
D. 1 3 2 6 8 5 9 7 4

10. 〔15-16〕 Insert $\{3, 8, 9, 1, 2, 6\}$ one by one into an initially empty BST. The post-order traversal sequence of the resulting tree is:

A. 2,1,3,6,9,8
B. 1,2,8,6,9,3
C. 2,1,6,9,8,3
D. 1,2,3,6,9,8

11. 〔15-16〕 Given a quadtree (四叉树) with 3 nodes of degree 2, 2 nodes of degree 3, and 4 nodes of degree 4. The number of leaf nodes is:

A. 10
B. 12
C. 20
D. 21

[Programming (PTA function problems, implement yourself)]

- 〔23-24〕 **Height of BST**: `int GetHeight(BinarySearchTree root);` plus `void GetPath(...)` — given the preorder input of a BST, return its height and output all longest paths.

- 〔24-25〕 **Find first node with a right child**: `int FirstHaveRight(int a[], int n);` — in a complete-binary-tree array (0 means missing), find the first node having a right child during an inorder traversal (iterative inorder).
- 〔15-16〕 **CheckBST**: `int CheckBST(BinTree T, int K);` — if T is a BST return the K-th smallest key, otherwise return the negative of the tree height.
- 〔15-16〕 **IsBalanced**: `bool IsBalanced(BinTree T, int *pHeight);` — judge whether a binary tree is height-balanced ($|H_L - H_R| \le 1$ at every node) and return its height.

4 专题四　优先队列与堆（Priority Queues / Heaps）

4.1 核心知识点

1. 优先队列 ADT (Priority Queue)

支持 `Insert`、`DeleteMin`（删最小）、`FindMin`。简单实现总有一端慢：无序表 Insert $O(1)$ / DeleteMin $O(N)$；有序表 Insert $O(N)$ / DeleteMin $O(1)$；**二叉堆 (binary heap)** 二者皆 $O(\log N)$。

2. 二叉堆 (Binary Heap)

两条性质 + 数组下标

结构性质 (structure property)：完全二叉树 (complete binary tree)。
堆序性质 (heap order property)：min-heap 中每个结点 $\le$ 其孩子（max-heap 相反）。
数组表示（下标从 1 开始）：结点 i 的父 $\lfloor i/2 \rfloor$，左孩子 $2i$，右孩子 $2i+1$；含 N 结点高度 $\lfloor \log_2 N \rfloor$。

3. 基本操作

- **Insert（上滤 percolate up）**：新元素置末尾后上移，$O(\log N)$。
- **DeleteMin（下滤 percolate down）**：用末尾元素填根后下移（每层选较小孩子），$O(\log N)$。
- **BuildHeap**：自底向上对 $\lfloor N/2 \rfloor, \ldots, 1$ 依次下滤，$\boxed{O(N)}$（非 $O(N \log N)$）。
- `DecreaseKey` 上滤、`IncreaseKey` 下滤、`Delete`=DecreaseKey 到 $-\infty$ 再 DeleteMin。

4. d-堆 (d-heap) 与第 k 大

d-heap 每结点 d 孩子，高 $O(\log_d N)$，DeleteMin 每层 $d-1$ 次比较。**第 k 大 (k-th largest)**：排序 $O(N \log N)$；大小为 k 的 min-heap 扫描 $O(N \log k)$；Quickselect 平均 $O(N)$。

易错点

- 堆只保证**父子**有序：min-heap 中“层 i 元素 $<$ 层 j 元素（$i < j$）”**错**（除非前者是后者祖先）。
- 堆的**中序遍历不是有序**的（heap $\neq$ BST）。
- min-heap 中**最大键必在叶子**（下标 $\lfloor N/2 \rfloor + 1, \ldots, N$）。
- `BuildHeap` 是 $O(N)$，逐个 Insert 才是 $O(N \log N)$。

4.2 真题巩固

真题巩固 Exercises（英文原题，无答案）

[True or False]

1. 〔24-25〕 Given a min-heap with unique keys, if $i < j$, then the element p in level i must be smaller than q in level j. (T **F**)

2. 〔23-24〕 In a min-heap, all the keys along the path from the root to any leaf node must be in non-decreasing order. (**T** F)

3. 〔20-21〕 The inorder traversal sequence of any max-heap must be in sorted order. (T **F**)

[Single Choice]

1. 〔24-25〕 The level-order traversal sequence of a min-heap is $\{5, 18, 29, 23, 64, 47, 36\}$. After calling DeleteMin twice, what is the post-order traversal of the resulting heap (after adjustment)?

A. 47,36,64,23,29
B. 47,64,36,23,29
C. 47,64,29,36,23
D. 47,64,36,29,23

2. 〔24-25〕 A 3-heap is a heap whose nodes have 3 children. In a min-3-heap with 2025 elements, which one of the following indices is NOT a possible position for the maximum key?

A. 675
B. 676
C. 1012
D. 1013

3. 〔24-25〕 Given that the key sequence $\{7, 9, 14, 19, 25, 20, 16, 21\}$ is a min-heap, after inserting key 2 and adjusting, the resulting min-heap is:

A. $\{2, 14, 7, 9, 25, 20, 16, 21, 19\}$
B. $\{2, 7, 14, 19, 20, 16, 21, 9, 25\}$
C. $\{2, 9, 14, 7, 20, 16, 21, 25, 19\}$
D. $\{2, 7, 14, 9, 25, 20, 16, 21, 19\}$

4. 〔23-24〕 Which of the following statements about d-heaps is TRUE?

A. In a d-heap, the parent of a given node can be found using integer division
B. In a complete d-heap, the number of leaf nodes is less than the number of internal nodes
C. D-heaps are used for sorting data in ascending order
D. The height of a d-heap with n elements is directly proportional to n

5. 〔23-24〕 The level-order sequence of a min-heap is $\{3, 12, 7, 53, 32, 8, 19\}$. Use the linear algorithm to adjust it into a max-heap, then call DeleteMax. The inorder traversal of the resulting tree is:

A. 32,12,7,3,19,8

B. 7,3,12,8,19,32

C. 3,8,12,7,19,32

D. 7,12,3,32,8,19

6. 〔18-19〕 The level-order sequence of a max-3-heap (each node has 3 children) is $\{88, 76, 65, 82, 68, 46, 52, 44, 62,$ Use the linear algorithm to adjust it into a min-3-heap, then run DeleteMin. How many nodes are NOT moved during the process?

A. 2

B. 3

C. 4

D. 5

7. 〔16-17〕 Insert $\{6, 15, 3, 9, 7, 4, 12, 10, 15, 14, 5, 13\}$ one at a time into an initially empty binary min-heap. After three DeleteMin operations, then inserting 8 and 11, the top element of the heap must be:

A. 3

B. 5

C. 6

D. 11

8. 〔15-16〕 Use the linear algorithm to adjust $\{28, 12, 7, 8, 19, 20, 15, 22\}$ into a max-heap, then insert 29. The resulting sequence is:

A. $\{29, 28, 20, 22, 12, 7, 15, 8, 19\}$

B. $\{29, 28, 20, 22, 19, 7, 15, 8, 12\}$

C. $\{29, 28, 22, 20, 19, 15, 12, 8, 7\}$

D. $\{7, 8, 28, 12, 19, 20, 15, 22, 29\}$

9. 〔15-16〕 In a max-heap with $n\,(>1)$ elements, the array index of the minimum key may be:

A. 1

B. $\lfloor n/2 \rfloor - 1$

C. $\lfloor n/2 \rfloor$

D. $\lfloor n/2 \rfloor + 2$

5 专题五　排序（Sorting）

5.1 核心知识点

1. 基本概念

- **稳定性 (stability)**：相等元素排序后相对次序不变即稳定 (stable)。
- **逆序对 (inversion)**：$i < j$ 但 $A[i] > A[j]$。插入排序时间 $= O(N + \#\text{inversions})$，故**接近有序**时很快。
- **比较排序下界**：平均 / 最坏均为 $\Omega(N \log N)$（决策树 decision tree 有 $N!$ 叶，高 $\geq \log_2 N!$）。

2. 各排序算法对比（必背）

算法	平均	最坏	空间	稳定	备注
Insertion Sort	$O(N^2)$	$O(N^2)$	$O(1)$	稳定	近乎有序 / 小规模快
Selection Sort	$O(N^2)$	$O(N^2)$	$O(1)$	不稳定	比较多、交换少
Bubble Sort	$O(N^2)$	$O(N^2)$	$O(1)$	稳定	一趟后未必有元素归位
Shellsort	$\sim O(N^{1.5})$	取决增量	$O(1)$	**不稳定**	Hibbard 增量 $2^k - 1$
Heapsort	$O(N \log N)$	$O(N \log N)$	$\boxed{O(1)}$	不稳定	原地、无最坏退化
Mergesort	$O(N \log N)$	$O(N \log N)$	$O(N)$	**稳定**	递归 / 迭代；外排序
Quicksort	$O(N \log N)$	$\boxed{O(N^2)}$	$O(\log N)$	不稳定	最快通用；pivot 三数取中
Radix Sort (LSD)	$O(d(N+r))$	$O(d(N+r))$	$O(N+r)$	稳定	非比较；从**最低位**起

3. 要点速记

- **Shellsort**：对不同增量 (increment) 做插入排序，增量递减至 1。
- **Quicksort**：选基准 (pivot) → 分区 (partition) → 递归。基准取“三数取中 (median-of-three)”避免最坏；已排序 + 端点基准退化 $O(N^2)$。**Quickselect** 求第 k 小平均 $O(N)$。
- **Table Sort**：只重排下标，按**置换环 (permutation cycles)** 物理归位。
- **Bucket / Radix Sort**：非比较排序，突破 $\Omega(N \log N)$ 下界；LSD 从个位到高位依次分配收集。

易错点

- Heapsort 额外空间 $O(1)$（原地），**不是** $O(N)$；Mergesort 才需 $O(N)$。
- Quicksort **最坏** $O(N^2)$；但分区时左右指针遇等于 pivot 的键都停下，则全相等输入恰为 $O(N \log N)$。
- Shellsort **不稳定**。
- “仅基于比较的排序，**最好**情况至少 $O(N \log N)$”**错**——Insertion Sort 最好 $O(N)$；$\Omega(N \log N)$

是平均 / 最坏下界。

- Radix Sort 从**最低有效位 (LSD)** 开始。

5.2　真题巩固

真题巩固 Exercises（英文原题，无答案）

[True or False]

1. 〔24-25〕 After performing one pass of radix sort on $\{11, 56, 25, 61, 79, 27, 51\}$, the result could be $\{11, 25, 27, 51, 56, 61, 79\}$. (T **F**)
2. 〔24-25〕 During the insertion sort process, the order of magnitude of the number of comparisons is independent of the initial state of the sequence. (T **F**)
3. 〔16-17〕 The best case time complexity of sorting algorithms based only on comparisons is at least $O(N \log N)$. (T **F**)
4. 〔16-17〕 To sort N records, heap sort requires at least $O(N)$ extra space. (T **F**)
5. 〔17-18〕 If there are less than 20 inversions in an integer array, then Insertion Sort will be the best method among Quick Sort, Heap Sort and Insertion Sort. (**T** F)
6. 〔17-18〕 After the first run of Bubble Sort, it is possible that no element is placed in its final position. (T **F**)
7. 〔18-19〕 Shell sort is stable. (T **F**)
8. 〔15-16〕 To sort N records by merge sort, the number of merge runs is $O(N \log N)$. (T **F**)

[Single Choice]

1. 〔24-25〕 Using direct insertion sort to sort the following four lists in ascending order, the one with the **fewest comparisons** is:

   A. 67,13,20,55,40,27,6,31
   B. 6,13,27,20,40,31,55,67
   C. 13,20,6,27,31,67,55,40
   D. 55,31,40,27,6,13,67,20

2. 〔24-25〕 Which of the following is NOT a possible result after the second pass of quicksort?

   A. 3,7,11,9,13,17,19
   B. 3,17,11,13,9,7,19
   C. 7,3,11,9,17,13,19
   D. 9,3,7,11,17,13,19

3. 〔23-24〕 When sorting $\{80, 58, 88, 11, 7, 25, 64\}$ in ascending order by Quick Sort with Median-of-Three partitioning, after the first run the number of inversions will decrease by __.

A. 9
B. 3
C. 2
D. 5

4. 〔23-24〕 When sorting $\{76971, 19927, 31681, 98978, 19537, 40134, 65401, 60983, 92952, 83584\}$ in ascending order by Radix Sort, which result is IMPOSSIBLE?

A. $\{65401, 76971, 31681, 92952, 60983, 40134, 83584, 19927, 19537, 98978\}$ after the 2nd run of LSD Sort
B. $\{76971, 31681, 65401, 92952, 60983, 40134, 83584, 19927, 19537, 98978\}$ after the 1st run of LSD Sort
C. $\{19927, 19537, 31681, 40134, 60983, 65401, 76971, 83584, 92952, 98978\}$ after the 2nd run of MSD Sort
D. $\{19927, 19537, 31681, 40134, 65401, 60983, 76971, 83584, 98978, 92952\}$ after the 1st run of MSD Sort

5. 〔17-18〕 To sort $\{8, 3, 9, 11, 2, 1, 4, 7, 5, 10, 6\}$ by Shell Sort, if we obtain $(4, 2, 1, 8, 3, 5, 10, 6, 9, 11, 7)$ after the first run, and $(1, 2, 3, 5, 4, 6, 7, 8, 9, 11, 10)$ after the second run, then the increments of these two runs must be:

A. 3 and 1
B. 3 and 2
C. 5 and 2
D. 5 and 3

6. 〔18-19〕 For the quicksort implementation where both the left and right pointers stop when an element with the same key as the pivot is found during partitioning, what is the running time when all keys are equal?

A. $O(\log N)$
B. $O(N)$
C. $O(N \log N)$
D. $O(N^2)$

7. 〔16-17〕 To sort N records iteratively by merge sort, the number of runs is:

A. $O(\log N)$
B. $O(N)$
C. $O(N \log N)$
D. $O(N^2)$

8. 〔15-16〕 If the input is a set of 10^5 1-digit numbers, which method can sort them in $O(N)$ time?

A. quick sort
B. insertion sort
C. bucket sort
D. Shell sort

9. 〔15-16〕 To sort $\{2, 12, 16, 88, 5, 10, 34\}$, the results of the first two runs are: after the 1st run $\{2, 12, 16, 10, 5, 34, 88\}$; after the 2nd run $\{2, 5, 10, 12, 16, 34, 88\}$. The possible sorting method is:

A. bubble sort
B. merge sort
C. insertion sort
D. quick sort

[Fill in the Blanks] 〔20-21〕 Shellsort with increment sequence $\{1, 3, 7, 11\}$. Fill in the two blanks.

```
void Shellsort( int A[], int N ) {
    int i, j, k, Increment, Inc[]={1,3,7,11};
    int Tmp;
    for ( k=3; k>=0; k-- ) {
        ________________ ;          /* blank 1: set increment of this pass */
        for ( i=Increment; i<N; i++ ) {
            Tmp = A[i];
            for ( j=i; j>=Increment; j-=Increment )
                if ( Tmp < A[j-Increment] )  A[j] = A[j-Increment];
                else  break;
            ________________ ;      /* blank 2: place Tmp */
        }
    }
}
```

6 专题六　散列表（Hashing）

6.1　核心知识点

1. 基本概念

哈希函数 (hash function) $f(x)$ 把键直接映射到桶下标，理想 $O(1)$。**装载因子 (loading factor)** $\lambda = \frac{\text{元素数}}{\text{表大小}}$。**冲突 (collision)**：不同键映到同一位置；**溢出 (overflow)**：桶满仍插入。

2. 哈希函数

整数：$f(x) = x \bmod \texttt{TableSize}$，**TableSize 取素数 (prime)**。字符串：移位散列（Horner 法则，×32 用左移 5 位）优于直接 ASCII 求和。

3. 冲突解决

方法	$f(i)$	λ 限制	聚集	删除
Separate Chaining	链表	≈ 1	无	直接删（简单）
Linear Probing	i	< 0.5	**主聚集 primary**	惰性删除
Quadratic Probing	i^2	< 0.5	次聚集 secondary	惰性删除
Double Hashing	$i \cdot h_2(x)$	< 0.5	无	惰性删除

平方探测定理 (Quadratic Probing)

若 **TableSize 为素数**且表**至少半空**（$\lambda < 0.5$），平方探测一定能插入成功。递推：$f(i) = f(i-1)+2i-1$。Double Hashing 推荐 $h_2(x) = R-(x \bmod R)$（R 为小于表长的素数，$h_2 \neq 0$）。

4. 惰性删除与再散列

开放地址法 (open addressing) **不能直接清空**已删槽：三态 `Empty / Legitimate / Deleted`，Find 遇 Deleted 须**继续探测** (lazy deletion)。**再散列 (rehashing)**：λ 超限或插入失败时建约 2 倍大素数新表重新插入；单次 $O(N)$，**摊还 (amortized)** $O(1)$。

易错点

- 平方探测“半满保证插入成功”依赖 **TableSize 为素数**；表长非素数（如 8）即使远未半满也可能插不进。
- **Separate Chaining** 当 $\lambda \to 1$ 时插入仍 $O(1)$（头插），**不会**严重变慢；变慢的是查找。
- 平方探测 $\neq$ 以 $h_2(k) = k$ 的 double hashing。
- 哈希表不支持有序操作，最坏可退化 $O(N)$。

6.2　真题巩固

真题巩固 Exercises（英文原题，无答案）

[True or False]

1. 〔24-25〕 If quadratic probing is used in hashing, then a new element can always be inserted if the hash table is at least half empty. (T　F)

2. 〔23-24〕 In hashing with quadratic probing to solve collisions, it is possible that a new element cannot be inserted if the table size is 8 and 3 cells are occupied. (T　F)

3. 〔17-18〕 If 7 elements have been stored in a hash table of size 13 at positions $\{0, 1, 2, 4, 5, 10, 11\}$ with $H(x) = x\%13$, then an empty spot can't be found when inserting 40 with quadratic probing. (T　F)

4. 〔18-19〕 Quadratic probing is equivalent to double hashing with a secondary hash function of $\text{Hash}_2(k) = k$. (T　F)

5. 〔20-21〕 In hashing, when the loading density approaches 1, the operation INSERTION will be seriously slowed down if the separate chaining method is used. (T　F)

6. 〔15-16〕 In hashing, functions "insert" and "find" have the same time complexity. (T　F)

[Single Choice]

1. 〔24-25〕 Which of the following statements about hashing is incorrect?

A. With quadratic probing, a new element can definitely be inserted if the table size is 14 and 7 cells are occupied.

B. If an insertion into the hash table fails, rehashing is necessary, and the new table size is about twice the original size.

C. If in double hashing $\text{Hash}_2(k) = k^2$, then double hashing effectively becomes quadratic probing.

D. The expected number of probes for insertions is equal to that of unsuccessful searches in linear probing.

2. 〔24-25〕 Insert $\{26, 30, 13, 17, 40, 21, 22, 10\}$ one by one into an initially empty hash table of size 11 with $H(\text{Key}) = \text{Key}\%11$, using linear probing. After insertion, what is the expected number of probes for an unsuccessful search?

A. $\frac{28}{11}$

B. $\frac{25}{11}$

C. $\frac{27}{11}$

D. $\frac{24}{11}$

3. 〔23-24〕 A hash table of size 13 with $H(\text{Key}) = \text{Key}\%13$ uses quadratic probing $h_i(k) = (H(k) \pm i^2)\%13$. After inserting $\{20, 6, 2, 16, 27, 15\}$ one by one, the address of 15 is:

A. 11
B. 6
C. 4
D. 2

4. 〔18-19〕 Insert $\{18, 23, 4, 26, 31, 33, 17, 39\}$ one by one into an initially empty hash table of size 13 with $H(\text{Key}) = \text{Key}\%13$ and linear probing. What is the loading density when the first collision occurs?

A. 0.54
B. 0.63
C. 0.31
D. 0.62

5. 〔17-18〕 Rehashing is required when an insertion to a hash table fails. Which of the following is NOT necessary for rehashing?

A. change the collision resolution strategy
B. use a new function to hash all the elements into the new table
C. build another table that is bigger than the original one
D. scan down the entire original hash table for non-deleted elements

6. 〔17-18〕 If the hash values of n keys are all the same, and linear probing is used to resolve collisions, then the minimum total number of probings to insert these n keys must be:

A. $n(n-1)/2$
B. $2n+1$
C. $2(n+1)$
D. $n(n+1)/2$

7. 〔15-16〕 What is a collision in hashing?

A. Everything else is the same for the two elements except their keys
B. Two elements have different data but share the same key
C. Two elements with different keys share the same hash value
D. Two elements with different hash values share the same key

[Fill in the Blanks] 〔23-24〕 Hashing for nonnegative numbers with rehashing (linear probing; rehash when the table is over half full). Fill in the two blanks.

```
void insert(HashTable *ht, int key) {
    if ( _________________ ) {            /* blank 1: need to rehash? */
```

```
        rehash(ht);
    }
    int index = hash_function(key, ht->size);
    while (ht->table[index] != -1) {
        index = _________________ ;       /* blank 2: linear probing next */
    }
    ht->table[index] = key;
    ht->count++;
}
```

7 专题七　并查集（Disjoint Set / Union-Find）

7.1 核心知识点

1. 等价关系 (Equivalence Relation)

关系 $\sim$ 是等价关系 $\iff$ 满足**自反 (reflexive)**、**对称 (symmetric)**、**传递 (transitive)**，它把集合划分为互不相交的**等价类 (equivalence class)**。

2. 森林表示与操作

每个集合是一棵树，孩子指向父，根代表集合；数组 `S[x]` 存父结点，根处存 $-$size（或 $-$height）。

- `Find(x)`：沿父指针上溯到根。　`Union`：把一棵树根挂到另一棵根下。
- 朴素合并可能形成长链，使一串操作达 $\Theta(N^2)$。

3. 优化

两大优化

按大小 / 高度合并 (union-by-size / height)：小树挂大树下 $\Rightarrow$ 树高 $\le \lfloor \log_2 N \rfloor$。
路径压缩 (path compression)：Find 时把路径上结点直接挂到根。
两者并用：M 次操作 $O(M\,\alpha(M,N))$，α 是反 Ackermann 函数 (inverse Ackermann)，实践中近似常数。

易错点

用 union-by-size / height 建的 N 结点树，高度**不会超过** $\lfloor \log_2 N \rfloor$；路径压缩会改变真实高度，故实际用 rank 作估计。

7.2 真题巩固

真题巩固 Exercises（英文原题，无答案）

[True or False]

1. 〔24-25〕 Suppose each single node in a disjoint set initially has height 0. The height of the resulting disjoint-set tree formed by performing union-by-height on n nodes may exceed $\lfloor \log_2 n \rfloor$. (T　F)

[Single Choice]

1. 〔24-25〕 The array representation of disjoint sets is $\{3, 3, 4, 7, 4, -3, -6, 6, 8, -2, 10\}$ (elements numbered 1 to 11, depth of the root is 0). After invoking `Union(Find(9),Find(11))`, `Union(Find(11),Find(3))` with union-by-size and path compression, what is the maximum depth of all the nodes?

A. 1

B. 2

C. 3

D. 4

2. 〔23-24〕 The array representation of disjoint sets is $S = \{-4, 0, 0, 2, -3, 4, 4, -1\}$ (elements numbered 0 to 7). Which operations will turn the array into $S = \{-5, 0, 0, 0, -3, 4, 4, 0\}$?

A. `Union(0,7); Find(6)`

B. `Find(5); Union(0,7)`

C. `Union(Find(3),7)`

D. `Find(3); Union(4,7)`

3. 〔16-17〕 The array representation of disjoint sets is $\{2, -4, 2, 3, -3, 5, 6, 9, -2\}$ (elements numbered 1 to 9). After invoking `Union(Find(4),Find(6))` with union-by-size, which elements will be changed in the resulting array?

A. elements of index 2 and 5

B. elements of index 2 and 6

C. elements of index 4 and 5

D. elements of index 4 and 6

4. 〔18-19〕 Let T be a tree created by union-by-size with N nodes. Then the height of T can be:

A. at most $\log_2 N + 1$

B. at least $\log_2 N + 1$

C. as large as N

D. anything that is greater than 1

5. 〔17-18〕 A relation R is defined on a set S. If for every element e in S, "eRe" is always true, then R is said to be __ over S.

A. consistent

B. symmetric

C. transitive

D. reflexive

6. 〔20-21〕 Given $S = \{1, \ldots, 11\}$ and 8 equivalence relations: $5 \sim 6, 7 \sim 8, 9 \sim 10, 2 \sim 6, 3 \sim 8, 6 \sim 8, 1 \sim 8, 1 \sim 5$. After applying them with union-by-size (if sizes are equal, the smaller element is the root) and path compression, which statement is FALSE?

A. there are 4 equivalence classes

B. element 1 is a child of element 5

C. all elements are either a root or a child of a root

D. element 2 and element 8 are siblings

8 专题八　图（Graph Algorithms）

8.1　核心知识点

1. 基本定义

- 完全图 (complete graph) 边数：无向 $\dfrac{N(N-1)}{2}$，有向 $N(N-1)$。
- 度 (degree)：无向 $\sum \deg(v) = 2|E|$；有向 $\sum$ in-degree $=\sum$ out-degree $= |E|$。
- **连通分量 (connected component)**；有向图**强连通 (strongly connected)**（任意两点互达）；N 顶点含 k 个分量时至少 $N-k$ 条边。

2. 存储表示

	空间	擅长
Adjacency Matrix	$O(V^2)$	判边、求**入度**（列求和）；稠密图
Adjacency List	$O(V+E)$	遍历邻接点、求**出度**；稀疏图

3. 拓扑排序 (Topological Sort，AOV 网络)

对 DAG，反复取**入度为 0 (in-degree 0)** 的顶点输出并删其出边，$O(V+E)$。拓扑序**唯一** $\Longleftrightarrow$ 每步恰好只有一个入度 0 顶点。有环则无拓扑序；DFS 逆后序亦可得拓扑序。

4. 最短路径 (Shortest Path)

场景	算法	复杂度 / 要点
无权 (unweighted)	BFS	$O(V+E)$
非负权	Dijkstra	邻接矩阵 $O(V^2)$；堆 $O(E\log V)$
含负权	队列式松弛	可判负环 (negative cycle)
DAG	按拓扑序松弛	$O(V+E)$；AOE 关键路径

Dijkstra 贪心：每次从未知集合选 `dist` 最小者确定，再**松弛 (relax)** 其邻接点。**AOE 网络**求最早 / 最晚发生时间 EC / LC，时差为 0 的活动构成**关键路径 (critical path)**。

5. 网络流与最小生成树

- **最大流 (Max Flow)**：不断找**增广路 (augmenting path)** 增流，在**残量图 (residual graph)** 中加反向边；**最大流 = 最小割 (max-flow min-cut)**。
- **最小生成树 (MST)**：**Prim**（从一点长树，选连接树与非树的最小边，**稠密图**佳）；**Kruskal**（按边权升序选边，用并查集判环，**稀疏图**佳）。**边权互异** $\Rightarrow$ **MST 唯一**。

6. DFS 应用

连通分量；**割点 / 双连通 (articulation point / biconnectivity)**：DFS 树记 `Num` 与 `Low`，根为割点 $\iff$ 有 ≥ 2 棵 DFS 子树，非根 u 为割点 $\iff$ 存在子结点 v 使 $\mathrm{Low}(v) \geq \mathrm{Num}(u)$；**欧拉回路 (Euler circuit)**。

易错点

- **Euler circuit** 经过每条**边**一次（非每个顶点）；无向连通图存在 $\iff$ 所有顶点**偶度**；恰 2 个奇度顶点 $\Rightarrow$ Euler path（非回路）。有向图：弱连通且每顶点**入度 = 出度**。
- “有向图每个顶点都在某环中” $\neq$ 强连通。
- 所有边权**同时** +1：最短路可能**改变**（边数多者增得多）；MST 结构不变，总权增加 $|V|-1$（不是 $|E|-1$）。
- Dijkstra **不能**处理负权边。　$|V| \geq 2$ 连通图至多 $|V|-2$ 个割点。

8.2　真题巩固

真题巩固 Exercises（英文原题，无答案）

[True or False]

1. 〔24-25〕 To find a minimum spanning tree in a dense graph, Prim's algorithm is more suitable than Kruskal's. (**T** F)

2. 〔24-25〕 In an undirected graph, a vertex v is an articulation point iff there exists a pair of vertices u and w (both different from v) so that every path between u and w passes through v. (**T** F)

3. 〔23-24〕 A graph with $|V|$ nodes ($|V| \geq 2$) has at most $|V|-2$ articulation points. (**T** F)

4. 〔23-24〕 Given a graph with $|V|$ nodes and $|E|$ edges, the total edge weight of its MST is W. If the weight of each edge is increased by 1, then W is increased by $|E|-1$. (T **F**)

5. 〔18-19〕 For a graph, if each vertex has an even degree, we can find an Euler circuit that visits every vertex exactly once. (T **F**)

6. 〔18-19〕 Let P be the shortest path from S to T. If the weight of every edge in the graph is incremented by 1, P will still be the shortest path from S to T. (T **F**)

7. 〔20-21〕 The storage size of a graph using the adjacency matrix is only related to the number of vertices but has nothing to do with the number of edges. (**T** F)

8. 〔16-17〕 If in a directed graph every vertex is part of some cycle, then the graph must be strongly connected. (T **F**)

9. 〔15-16〕 In a directed graph, the sum of the in-degrees must be equal to the sum of the out-degrees of all the vertices. (**T** F

）

[Single Choice]

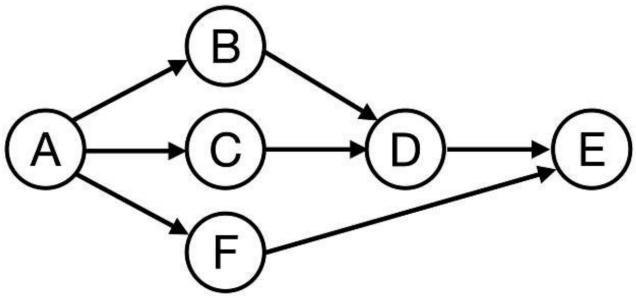
1. 〔24-25〕 Using Dijkstra's algorithm with a min-priority queue to compute shortest paths, what is the total time spent on the relaxation step "decrease the distance of each vertex adjacent from the current visited one"?

A. $O(|E|)$
B. $O(|V| \cdot |E|)$
C. $O(|V| \log |V|)$
D. $O(|E| \log |V|)$

2. 〔24-25〕 Given a weakly-connected *directed* graph, what condition ensures that an Eulerian Circuit exists?

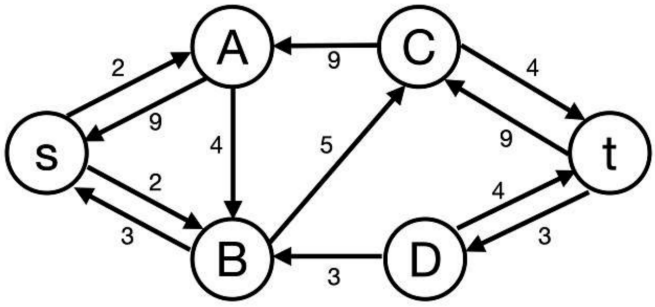
A. All vertices have equal in-degree and out-degree.
B. Exactly one vertex has out-degree = in-degree +1, one vertex has in-degree = out-degree +1, and all the rest have equal in-degree and out-degree.
C. Exactly two vertices have odd total-degree, and the rest have even total-degree.
D. All vertices have even total-degree.

3. 〔24-25〕 How many valid topological sorts does the following graph have?



A. 7
B. 8
C. 9
D. 10

4. 〔24-25〕 The following figure shows the residual graph after sending 12 units of flow from s to t. What is the maximum flow from s to t?



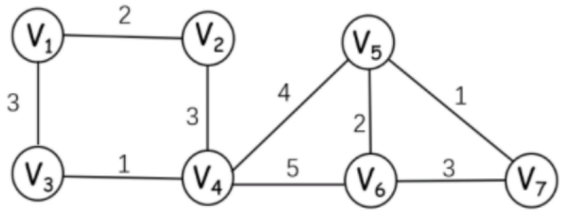
A. 12

B. 14

C. 16

D. 20

5. 〔23-24〕 The following undirected weighted graph grows its minimum spanning tree by Kruskal's algorithm. Which statement is correct?



A. (v_5, v_7) must be an edge of the MST, but (v_2, v_4) must not be.

B. (v_5, v_6) must be an edge of the MST, but (v_4, v_6) must not be.

C. (v_3, v_4) must be an edge of the MST, but (v_5, v_4) must not be.

D. (v_1, v_3) must be an edge of the MST, but (v_4, v_6) must not be.

6. 〔23-24〕 A directed graph is given by the adjacency list below. Which one is a valid topological order?

```
v0:null  v1:v0,v3  v2:v1  v3:null  v4:v1,v6  v5:v2,v4  v6:null  v7:v1,v2
```

A. 7 5 2 4 6 1 3 0

B. 7 5 2 1 0 3 4 6

C. 6 0 3 1 4 2 7 5

D. 5 7 4 1 3 0 2 6

7. 〔23-24〕 Which algorithms can be used to solve the single source shortest path problem for an unweighted DAG? I. Kruskal II. Breadth-first search III. Topological Sort IV. Dijkstra

A. II and IV only

B. I and IV only

C. All of them

D. II, III and IV only

8. 〔20-21〕 In a weighted undirected graph, if the length of the shortest path from v_1 to v_0 is 14, and there exists an edge of weight 1 between v_2 and v_1, then which is correct?

A. The shortest path from v_2 to v_0 must be no greater than 13.

B. The shortest path from v_2 to v_0 must be no less than 13.

C. The shortest path from v_2 to v_0 must be less than 15.

D. The shortest path from v_2 to v_0 must be greater than 15.

9. 〔18-19〕 A graph with 100 vertices and 12 edges must have at most how many connected components?

A. 87

B. 88

C. 94

D. 95

10. 〔15-16〕 If graph G is NOT connected and has 21 edges, then it must have at least how many vertices?

A. 7

B. 8

C. 9

D. 10

11. 〔15-16〕 To check if there is a cycle in a directed graph, which algorithm can be used besides topological sort?

A. Prim

B. BFS

C. DFS

D. Dijkstra

[Programming (PTA function problems, implement yourself)]

- 〔24-25〕 **Finding Independent Set**: `int Independent_Set(MGraph G, int S[]);` — greedily pick the unvisited vertex with the smallest index into S and delete all its adjacent vertices.
- 〔18-19〕 **Is Topological Order**: `bool IsTopSeq(LGraph Graph, Vertex Seq[]);` — test whether `Seq` is a topological order of the graph.
- 〔17-18〕 **Shortest Distance**: `void ShortestDist(MGraph Graph, int dist[], Vertex S);` — weighted shortest distance from each vertex to source S (positive weights); store -1 if unreachable.

- 〔23-24〕 **Eulerian**: test whether a connected undirected graph has an Euler tour, an Euler circuit, or is not Eulerian (count odd-degree vertices).

——讲义完——祝考试顺利！